# AI ASSISTED CODING

**Name:** P.Swaran Raj **Roll.No:** 2303A52247
**Batch:** 37 **Assignment-** 13.2

**Task Description -1 (Refactoring – Eliminating Repetitive Logic)**
• **Task:** Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

**Prompt:**
**"# for the below code locate the repeated logics and encapsulate them into a reusable function and add proper doc strings to the function. And the program output should remain unchanged after refactoring"**

AI Assit 13.2.py > calculate_area_and_perimeter

```python
# for the below code locate the repeated logics and encapsulate them into a resuable functic
def calculate_area_and_perimeter(length, width):
    """
    Calculate the area and perimeter of a rectangle given its length and width.

    Parameters:
    length (float): The length of the rectangle.
    width (float): The width of the rectangle.

    Returns:
    tuple: A tuple containing the area and perimeter of the rectangle.
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
area, perimeter = calculate_area_and_perimeter(length, width)
print("Area:", area)
print("Perimeter:", perimeter)

length = 12
```

Ln 15, Col 27 Spaces: 4 UTF-8 CRLF { } Python 3.13.3 Python 3.13 (64-bit)

Search ENG IN 09:43 10-03-2026

AI Assit 13.2.py > ...

```python
def calculate_area_and_perimeter(length, width):

    Parameters:
    length (float): The length of the rectangle.
    width (float): The width of the rectangle.

    Returns:
    tuple: A tuple containing the area and perimeter of the rectangle.
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter
length = 8
width = 4
area, perimeter = calculate_area_and_perimeter(length, width)
print("Area:", area)
print("Perimeter:", perimeter)

length = 12
width = 6
area, perimeter = calculate_area_and_perimeter(length, width)
print("Area:", area)
print("Perimeter:", perimeter)
```

Ln 1, Col 204 Spaces: 4 UTF-8 CRLF { } Python 3.13.3 Python 3.13 (64-bit)

Search ENG IN 09:50 10-03-2026

## Explanation:

The AI identified repeated calculations for area and perimeter.
The logic was moved into a reusable function to improve maintainability.

## Output:

```
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
```

The output remained unchanged even after Refactoring the program.

## Observation:

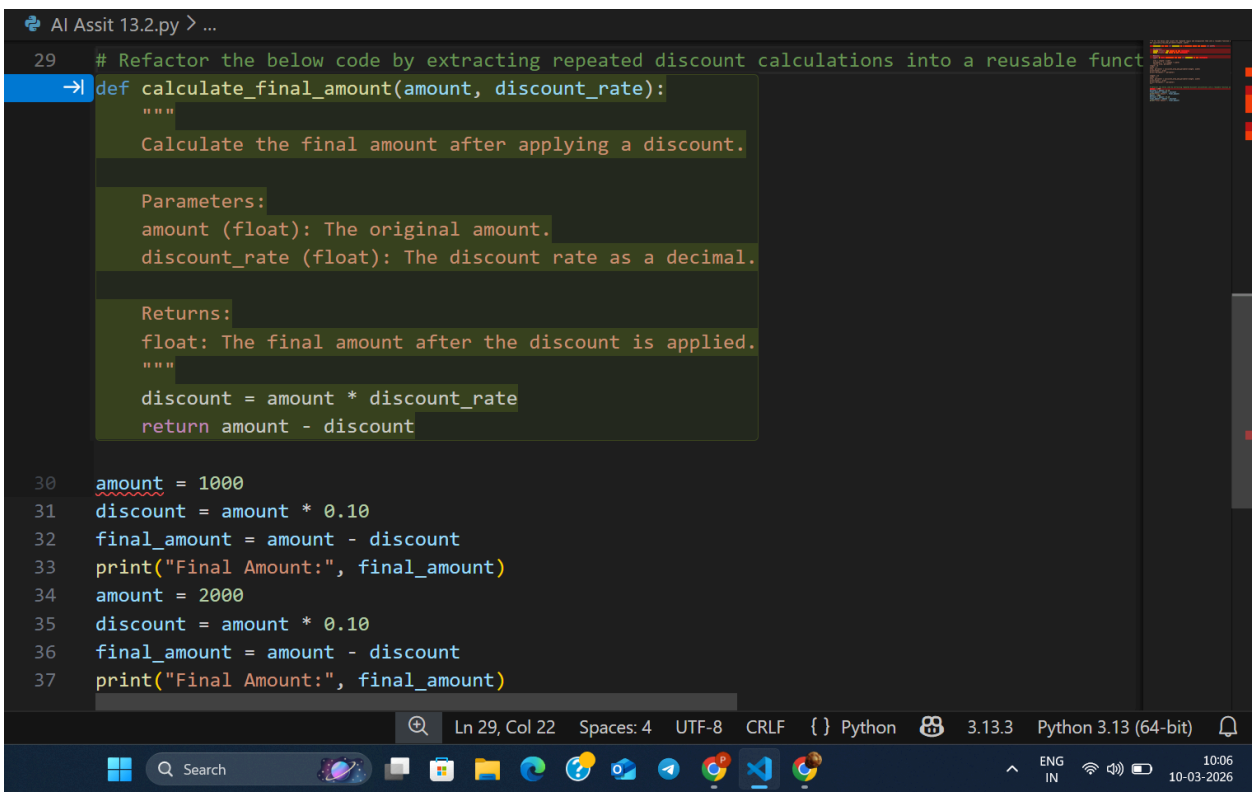
Refactoring removed duplicated code and improved readability and reuse.

**Task Description -2 (Refactoring – Modularizing Inline Calculations)**

• **Task:** Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

**Prompt:**

**“# Refactor the below code by extracting repeated discount calculations into a reusable function and ensure code readability with proper docstrings.”**



AI Assit 13.2.py > ...

```python
# Refactor the below code by extracting repeated discount calculations into a reusable funct
def calculate_final_amount(amount, discount_rate):
    """
    Calculate the final amount after applying a discount.

    Parameters:
    amount (float): The original amount before discount.
    discount_rate (float): The discount rate to be applied (e.g., 0.10 for 10%).

    Returns:
    float: The final amount after applying the discount.
    """
    discount = amount * discount_rate
    final_amount = amount - discount
    return final_amount
amount = 1000
final_amount = calculate_final_amount(amount, 0.10)
print("Final Amount:", final_amount)
amount = 2000
final_amount = calculate_final_amount(amount, 0.10)
print("Final Amount:", final_amount)
```

Ln 3, Col 52 Spaces: 4 UTF-8 CRLF { } Python 3.13.3 Python 3.13 (64-bit)

Search ENG IN 10:09 10-03-2026

**Explanation:**

AI extracted the discount calculation logic into a separate function with clear documentation.

**Output:**

```
Final Amount: 900.0
Final Amount: 1800.0
```

Area and perimeter values are displayed correctly for both rectangles.

**Observation:**

Refactoring removed duplicated code and improved readability and reuse.

**Task Description -3 (Refactoring – Improving Loop and Conditional Performance)**

• **Task:** Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops and repeated conditional Checks.

**Prompt:**

**“#Analyze the following Python code and suggest a more efficient implementation by removing nested loops and improving lookup performance. Also compare execution time before and after refactoring. ”**

```python
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(name, "found")
    else:
        print(name, "not found")
# Analyze the following Python code and suggest a more efficient implementation by removing
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
student_set = set(students)  # Convert the list of students to a set for O(1) average time c
for name in check_list:
    if name in student_set:  # Check for membership in the set
        print(name, "found")
    else:
        print(name, "not found")
```

## Explanation:

AI suggested converting the student list into a set to improve search performance.

# Execution Time Comparison

| Version | Data Structure | Time Complexity |
|---|---|---|
| Original Code | List with nested loop | O(n × m) |

| | | |
|---|---|---|
| Optimized Code | Set lookup | O(n) |

The optimized version performs faster because **set membership checking is O(1)** on average.

**Output:**

Execution time of the optimized version is **lower compared to the original nested loop implementation**.

**Observation:**
AI-assisted refactoring identified inefficient nested loops and recommended using a set for faster lookups.This improved both code readability and execution efficiency while maintaining the same output.

**Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)**
• **Task:** Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.
**Prompt:**
**"#Refactor the code by replacing hardcoded values with named constants."**

```
print("Simple Interest:", (1000 * 2 * 5) / 100)
print("Simple Interest:", (2000 * 2 * 5) / 100)
# Refactor the code by replacing hardcoded values with named constants.
PRINCIPAL_1 = 1000
PRINCIPAL_2 = 2000
RATE = 5
TIME = 2
simple_interest_1 = (PRINCIPAL_1 * TIME * RATE) / 100
simple_interest_2 = (PRINCIPAL_2 * TIME * RATE) / 100
print("Simple Interest:", simple_interest_1)
print("Simple Interest:", simple_interest_2)
```

```
(constant) PRINCIPAL_1: Literal[1000]
```

```python
        print(name, "not found")'''

print("Simple Interest:", (1000 * 2 * 5) / 100)
print("Simple Interest:", (2000 * 2 * 5) / 100)
# Refactor the code by replacing hardcoded values with named constants.
PRINCIPAL_1 = 1000
PRINCIPAL_2 = 2000
RATE = 5
TIME = 2
simple_interest_1 = (PRINCIPAL_1 * TIME * RATE) / 100
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\pulla\OneDrive\Documents\SRU\AI Asisted Coding> & C:/Users/pulla/AppData/Loca
/Python313/python.exe "c:/Users/pulla/OneDrive/Documents/SRU/AI Asisted Coding/AI Assit 1
Simple Interest: 100.0
Simple Interest: 200.0
Simple Interest: 100.0
Simple Interest: 200.0
```

**Explanation:**
AI replaced numeric values with constants such as RATE and TIME for clarity.
**Output:**
Simple interest values are calculated correctly.

**Observation:**
Using constants improves readability and makes future changes easier.

**Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)**
• **Task:** Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments.

**Prompt:**
**"# Improve the above code with descriptive variable names and add comments for better readability."**

```python
x = 50
y = 30
z = x + y
print(z)
# Improve the above code with descriptive variable names and add comments for better readabi
first_number = 50
second_number = 30
sum_of_numbers = first_number + second_number
print(sum_of_numbers)
```

```python
x = 50
y = 30
z = x + y
print(z)
# Improve the above code with descriptive variable names
first_number = 50
second_number = 30
sum_of_numbers = first_number + second_number
print(sum_of_numbers)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\pulla\OneDrive\Documents\SRU\AI Asisted Coding> & C:/Use
n313/python.exe "c:/Users/pulla/OneDrive/Documents/SRU/AI Asisted Co
80
80
PS C:\Users\pulla\OneDrive\Documents\SRU\AI Asisted Coding>
```

## Explanation:

AI replaced unclear variable names with descriptive ones and added comments.

## Output:

The program correctly prints the sum of the two numbers.

**Observation:**

Descriptive variable names improve code readability and maintainability.

**Conclusion:**

This lab demonstrated how AI-assisted refactoring improves code readability, reduces duplication, and enhances program maintainability while preserving functionality.